Laboratory Work 1

DevOps Fundamentals: Linux, Git, Docker and Jenkins

Name: Zholaman

Surname: Bekbolatov

Group: IT2-2314

Student ID: 37614

Course: DevOps

Date: 19 October 2026

Repository: https://github.com/zen-vov/bekbolatov-zholaman-devops

Requirements source: Devops_Task_6week.docx. All 21 technical tasks were implemented and verified against actual command output and a successful Jenkins build. The student must separately record and submit the continuous personal video with their own voice.

Report contents: Linux; Git & GitHub; Docker; Jenkins; Integrated Workflow; task overview; conclusion and submission requirements.

1. Linux - Tasks 1-5

The Linux demonstration ran successfully in Ubuntu 24.04. Sources were mounted at /root/devops/bekbolatov_zholaman, preserving the required ~/devops/surname_name layout. Required app, scripts, config, logs and backup folders and personalized files were created.

linux-demo.sh executed pwd, ls, cd, mkdir, touch, cp, mv, rm, cat, head and tail. A ten-line sample log was searched with find and grep. Exactly two ERROR and two WARNING messages were present; the last five lines were displayed.

private.txt was set to 600, public.txt to 644 and the Bash script to 755. The script printed identity and system data with whoami, hostname, date, uname, df and free. Both existing-file and missing-file branches were checked. ps, ps aux and top exposed PID, owner, CPU and memory.

```
++ dirname scripts/linux-demo.sh
+ cd scripts/..
+ pwd
/root/devops/bekbolatov_zholaman
+ ls -la
total 16
drwxr-xr-x 10 root root  320 Oct 19 10:06 .
drwxr-xr-x  3 root root 4096 Oct 19 10:09 ..
-rw-r--r--  1 root root   84 Oct 19 10:06 Bekbolatov_Zholaman_info.txt
drwxr-xr-x  6 root root  192 Oct 19 10:08 app
drwxr-xr-x  2 root root   64 Oct 19 10:06 backup
drwxr-xr-x  3 root root   96 Oct 19 10:06 config
drwxr-xr-x  3 root root   96 Oct 19 10:06 logs
-rw-------  1 root root   29 Oct 19 10:06 private.txt
-rw-r--r--  1 root root   40 Oct 19 10:06 public.txt
drwxr-xr-x  4 root root  128 Oct 19 10:08 scripts
+ mkdir -p backup logs
+ printf '%s\n' 'INFO Application started' 'INFO User logged in' 'WARNING Configuration file missing' 'INFO Configuration
loaded' 'ERROR Connection failed' 'INFO Application restarted' 'WARNING Memory usage is high' 'INFO Request received' 'ERROR
Request failed' 'INFO Application stopped'
+ touch backup/demo.txt
+ cp Bekbolatov_Zholaman_info.txt backup/info-copy.txt
+ mv backup/demo.txt backup/moved.txt
+ rm backup/moved.txt
+ cat Bekbolatov_Zholaman_info.txt
Name: Zholaman
Surname: Bekbolatov
Group: IT2-2314
Student ID: 37614
Course: DevOps
+ head -n 3 logs/application.log
INFO Application started
INFO User logged in
WARNING Configuration file missing
+ tail -n 5 logs/application.log
INFO Application restarted
WARNING Memory usage is high
INFO Request received
ERROR Request failed
INFO Application stopped
+ find . -name application.log
./logs/application.log
```

Figure 1. Browser display of the actual Ubuntu command log (not an emulated terminal).

Full evidence: docs/evidence/linux.txt. Permission bits mean read, write and execute for owner, group and others; private notes allow only owner access, while public notes allow everyone to read.

2. Git & GitHub - Tasks 6-10

The existing repository was reused. Git identity was checked; actual implementation was recorded in meaningful commits. The development branch introduced Jenkins automation and was merged into main with a merge commit. Both branches were pushed to the existing origin; no force push or history deletion was performed.

The .gitignore excludes *.log, backup/, .env, node_modules and local Jenkins secrets. git check-ignore verified the rules. The public GitHub repository was verified through its API, and Jenkins subsequently checked out its main branch over HTTPS.

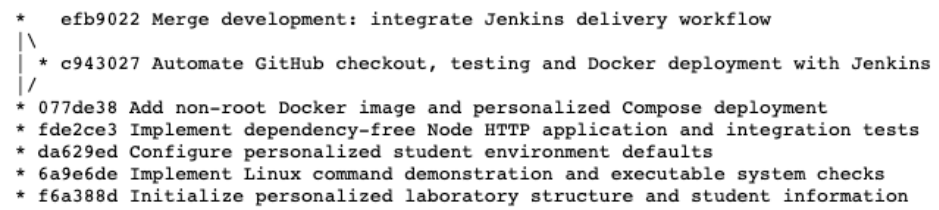


Figure 2. Actual git log --oneline --graph --all output viewed in the browser.

Evidence: docs/evidence/git-history.txt, gitignore.txt, github-refs.txt, github-repository.json and git-identity.txt. Git preserves history; GitHub makes the versioned repository accessible to Jenkins and the instructor.

3. Docker - Tasks 11-15

The Dockerfile uses FROM, WORKDIR, COPY, RUN, EXPOSE and CMD. The dependency-free Node HTTP server displays personal information at / and provides /health. It runs as the node user, stays alive to serve requests, and closes gracefully on SIGTERM.

A real image was built. The required container was started with all four student variables and verified through HTTP. A separate container execution asserted overrides of all four variables. docker logs, stop, start, exec and rm succeeded. Compose up --wait, application verification and Compose down succeeded.

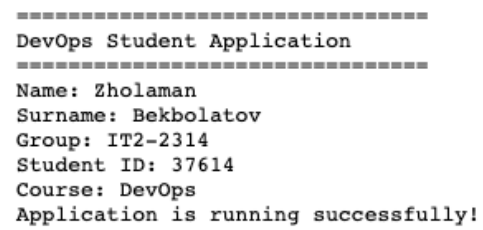


Figure 3. Personal information returned by the running Docker application at localhost:37614.

Evidence: docs/evidence/docker-build.txt, docker-lifecycle.txt and container-state.json. The final container state is running and healthy. An image is the reusable template; a container is its running instance. Environment variables configure a deployment without changing the image.

4. Jenkins - Tasks 16-19

Jenkins was deployed in its own Docker Compose stack at localhost:18080. Pipeline, Git and Stage View plugins were installed. An authenticated account and the exact job Bekbolatov_Zholaman_DevOps were configured. Local credentials are kept in an ignored file.

The job uses Pipeline script from SCM with the public GitHub repository, main branch and Jenkinsfile. Jenkins has Node/npm and Docker CLI. The local educational controller accesses the host Docker socket; its web port is bound to localhost.

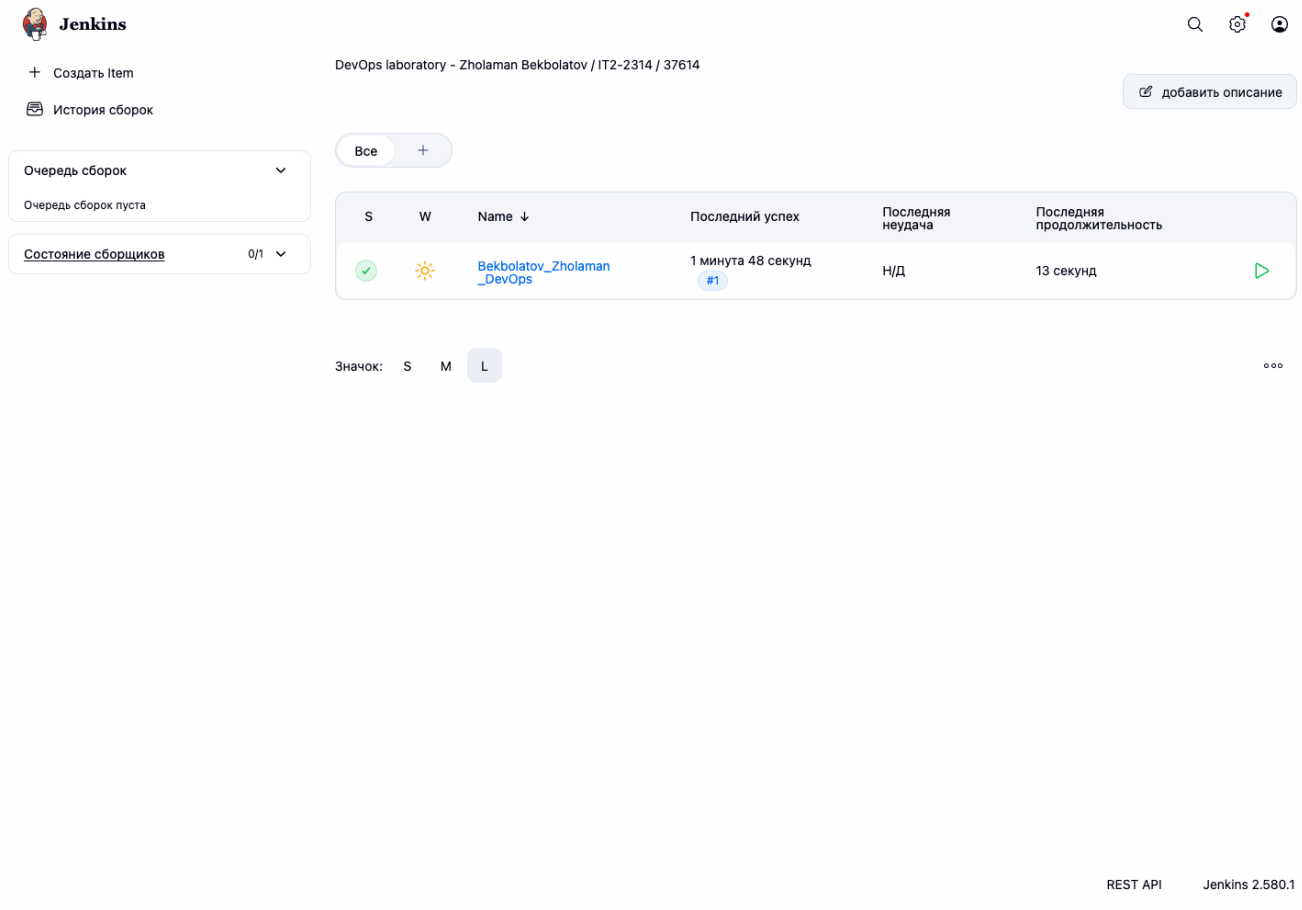


Figure 4. Running Jenkins Dashboard with the personalized job.

The Jenkinsfile sets STUDENT_NAME, STUDENT_SURNAME, STUDENT_GROUP and STUDENT_ID. Console Output displays Zholaman Bekbolatov, IT2-2314 and 37614. The Docker Run stage passes these variables to the container and validates health and student ID.

5. Pipeline stages - Tasks 17-20

Build #1 completed successfully. The stage API independently records SUCCESS for Checkout, Build, Test, Docker Build and Docker Run. The Node test runner passed all three tests, including real HTTP requests and all four environment overrides.

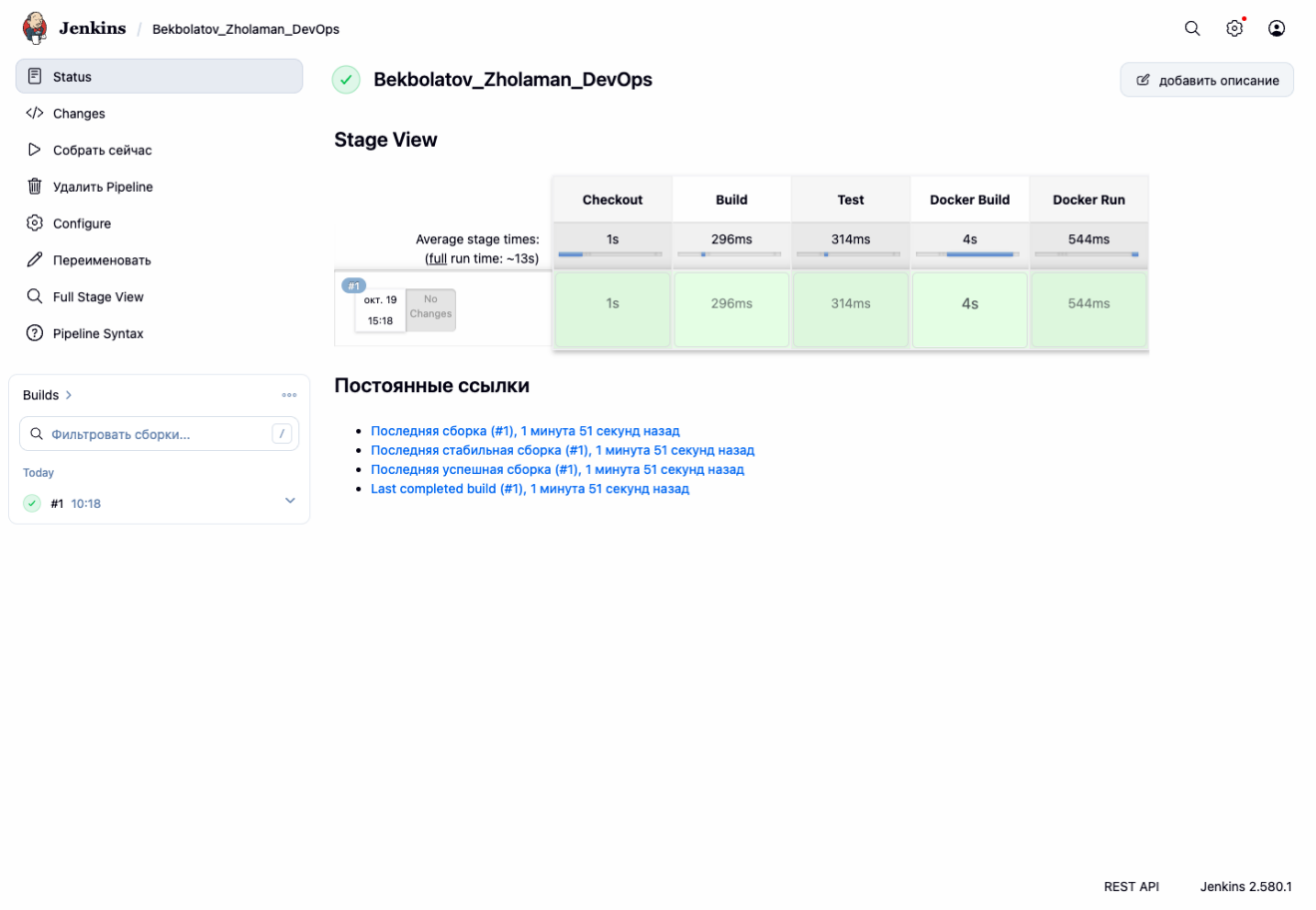


Figure 5. Real Jenkins Stage View: all five required stages are green.

Verified source commit: efb9022f72dfb62cddeef82a1997a9ff71212734

The source commit is the implementation merge. Later commits add documentation and evidence. The running application image is tagged bekbolatov-zholaman-devops:1 and latest. Evidence: jenkins-build.json, jenkins-stages.json, jenkins-console.txt and docker-running.txt.

6. Successful execution - Task 20

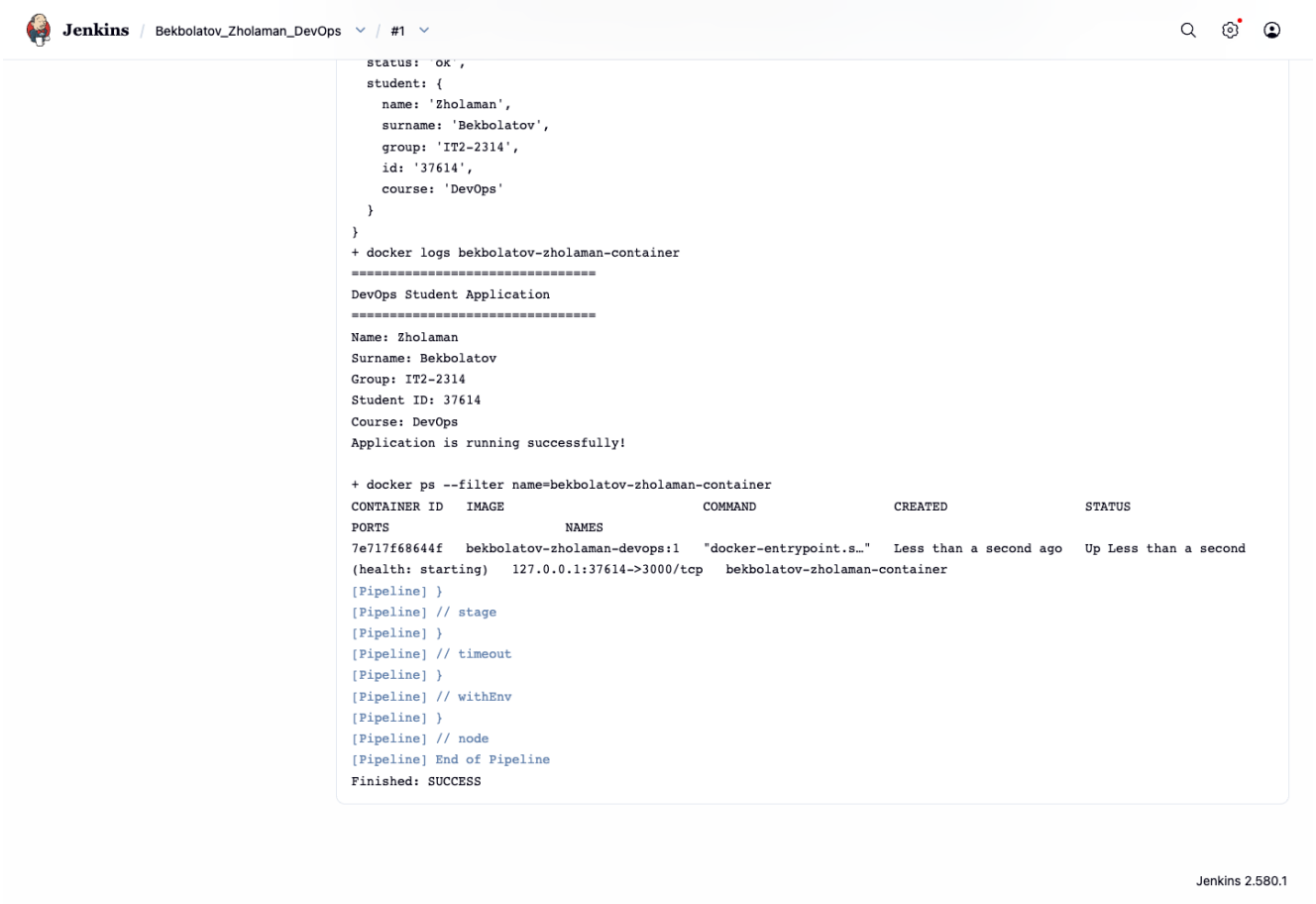


Figure 6. Actual Console Output: student data, Docker deployment and Finished: SUCCESS.

No local-source fallback was used: Jenkins first retrieved the Jenkinsfile from GitHub and then checked out the same repository in Checkout. The health request in Docker Run returned the configured student ID. The final container is healthy according to docker inspect.

All screenshots in this report were captured from a live Chromium browser. Text-log screenshots show the actual saved outputs, not recreated terminal graphics. Original images are in docs/screenshots; full outputs are in docs/evidence.

7. Integrated Workflow - Task 21

Linux creates and manages the project files and scripts. Git records the project changes. GitHub hosts the repository. Jenkins obtains the Jenkinsfile and a specific source revision from GitHub, builds and tests the application, creates its Docker image, and starts the container. The health check connects the successful build with a functioning application.

```
Linux -> Project -> Git -> GitHub -> Jenkins
          Checkout -> Build -> Test -> Docker Build -> Docker Run
          HTTP health verification -> Finished: SUCCESS
```

Linux provides shell and process tools; Git provides reproducible history; GitHub shares the source; Jenkins automates the ordered steps; Docker packages and executes the application. Compose describes reusable deployment configuration.

A failing stage stops the normal stages that follow and marks the build as failed. For example, a failed Test stage prevents building or running a new image. This simple pipeline does not promise automatic rollback; earlier deployments can remain or require correction and rerunning.

Run instructions are in README.md. Reproducible demonstrations are in bekbolatov_zholaman/scripts/linux-demo.sh and verify-docker.sh. The video link is at the top of README.md.

Official references: Jenkins Docker installation - https://www.jenkins.io/doc/book/installing/docker/ ; Jenkins Pipeline syntax - https://www.jenkins.io/doc/book/pipeline/syntax/.

8. Final checklist and conclusion

1. Project Environment	**Verified**
2. Search and File Processing	**Verified**
3. File Permissions	**Verified**
4. Bash Script	**Verified**
5. Processes	**Verified**
6. Git Repository	**Verified**
7. Meaningful Commit History	**Verified**
8. .gitignore	**Verified**
9. Branching and Merge	**Verified**
10. GitHub	**Verified**
11. Dockerfile	**Verified**
12. Build and Run Docker Container	**Verified**
13. Environment Variables	**Verified**
14. Container Management	**Verified**
15. Docker Compose	**Verified**
16. Jenkins Job	**Verified**
17. Jenkinsfile	**Verified**
18. Jenkins + GitHub	**Verified**
19. Jenkins Environment Variables	**Verified**
20. Successful Pipeline	**Verified**
21. Complete DevOps Workflow	**Verified**

All 21 technical tasks passed real verification. This does not constitute a guaranteed grade: the DOCX separately assesses repository organization, individual understanding and the personal video. The assessment totals 100 points, including 5 for video.

Submission still required: record one continuous screen video with your own voice, review its sound and content, and submit its link together with the GitHub repository URL and this PDF. No missing technical screenshots remain.

Keep Docker Desktop running for the live demonstration. Jenkins login: zholaman; read the password privately from jenkins/secrets/admin_password. Do not submit passwords or browser session artifacts.